□ LEOPARDO RH

DOSSIER 2 — PROMPTS D'EXÉCUTION & ORCHESTRATION IA

Qui fait quoi, quand, et avec quel prompt | Mars 2026

CLAUDE	JULES	CURSOR	VO.DEV	MANUS
Conception & Backend	Mobile Flutter	Frontend Vue.js	UI Components & Révision	Tâches Auditeur

1. QUI FAIT QUOI — TABLEAU DE RÉPARTITION DES RÔLES

AGENT IA	DOMAINE PRINCIPAL	NE TOUCHE PAS À
□ CLAUDE (claude.ai + Claude Code)	Conception, documentation, analyse, backend Laravel 11, API REST, SQL, migrations, tests Pest, seeders, TenantMiddleware, queue workers, PDF génération	Code Flutter, styles Vue.js UI
□ JULES (Google AI Code Assistant)	Application Flutter : screens, widgets, Riverpod providers, GoRouter, Dio interceptors, firebase_messaging, local_auth, geolocator, QR scanner	Code Laravel, base de données, déploiement
□ CURSOR (IDE AI)	Frontend Vue.js 3 + Inertia.js : composants, pages gestionnaire web, Tailwind CSS, PrimeVue. Refactoring backend sur demande.	Flutter, architecture backend
□ VO.DEV (Vercel v0)	Génération rapide de composants UI Vue.js ou React convertis ensuite. Landing page HTML/CSS. Prototypage visuel rapide.	Logique métier, base de données, API
□ MANUS (Agent autonome)	Tâches longues autonomes : générer tous les fichiers mock JSON, créer les 20 modèles Dart complets, générer le CHANGELOG, créer les fichiers YAML CI/CD.	Code métier critique, architecture

Règle d'or de la coopération IA

1. Le fichier 02_API_CONTRATS.md est la BIBLE commune. Tout ce que Claude Code produit comme API, Jules le consomme selon ce contrat. Si l'un change quelque chose, il met à jour le contrat ET en informe l'autre.

2. Jamais deux agents ne modifient le même fichier en même temps. CLAUDE possède le backend. JULES possède le mobile. CURSOR possède le frontend web.

3. Chaque agent reçoit son prompt complet (ci-dessous) avec tous les documents de référence. L'agent ne doit PAS improviser l'architecture.

2. CHRONOLOGIE — QUI INTERVIENT QUAND

PÉRIODE	AGENT	TÂCHE EXACTE
MAINTENANT	Claude (claude.ai)	Compléter API contrats (auth/me, attendance/today, fcm)
	Manus	Générer les 8 fichiers JSON mock + modèles Dart Employee et AttendanceLog
	Claude (claude.ai)	Créer README monorepo avec conventions Git
Sprint 0 Sem. 1	Toi + Claude.ai	Setup Firebase, Google Play, App Store Connect, domaine DNS, o2switch SSH
	Claude Code	Créer projet Laravel 11, installer packages, configurer PostgreSQL, Redis, Sanctum, TenantMiddleware, seeders
	Jules	Créer projet Flutter 3.x, configurer GoRouter + Riverpod + Dio, firebase_messaging, écran de login avec mock
Sem. 2-4	Claude Code	Auth module complet (login, logout, refresh, forgot password, TenantMiddleware), tests Pest isolation
	Jules	Écran pointage (check-in/out) + historique — avec mock JSON d'abord
	Manus	Compléter tous les modèles Dart (8 classes) + fichiers mock complets
Sem. 5-8	Claude Code	Modules absences, avances, tâches, paie — backend + tests
	Jules	Écrans absences, avances, tâches — Flutter — connecté à l'API réelle
	Cursor	Dashboard gestionnaire Vue.js — modules pointage + absences
Sem. 9-12	Claude Code	Module paie, PDF bulletins, rapports, évaluations, Super Admin
	Jules	Écrans paie, évaluations, profil — Flutter
	Cursor	Dashboard gestionnaire : paie, rapports, Super Admin interface
Sem. 13-16	v0.dev	Landing page commerciale + tunnel de vente HTML/CSS
	Cursor	Intégration landing page dans Blade Laravel (SEO)
	Claude Code	Deploy.yml + configuration o2switch + Supervisor + Nginx
	Manus	Génération CHANGELOG + validation finale de la documentation

3. PROMPTS D'EXÉCUTION — UN PAR AGENT PAR TÂCHE

□ CLAUDE CODE — Backend Laravel 11

PROMPT CC-01 — Initialisation Laravel (Sprint 0, Semaine 1)

```
Tu es l'architecte backend de Leopardo RH. Tu vas initialiser le projet Laravel
11.

STACK OBLIGATOIRE (non négociable) :
- Laravel 11 (PHP 8.3)
- PostgreSQL 16 multi-schéma (stancl/tenancy évalué, sinon TenantMiddleware
custom)
- Redis 7 (cache + queues Horizon)
- Sanctum (tokens par appareil)
- DomPDF (bulletins de paie PDF)
- Firebase (kreait/laravel-firebase pour push notifications)

ÉTAPES À EXÉCUTER DANS L'ORDRE :
1. composer create-project laravel/laravel leopardo-rh-api
2. Installer packages : sanctum, dompdf, firebase, pest, horizon, telescope
3. Configurer config/database.php pour PostgreSQL 16
4. Créer TenantMiddleware (voir 11_MULTITENANCY_STRATEGY.md)
5. Exécuter le schéma public (voir 07_SCHEMA_SQL_COMPLET.sql, section schéma
public)
6. Créer les seeders : langues, plans, modèles RH par pays (voir 05_SEEDERS.md)
7. Configurer .env.example avec toutes les variables nécessaires

RÉFÉRENCE OBLIGATOIRE : Lire 01_PROMPT_MASTER_CLAUDE_CODE.md en entier avant de
commencer.
RÈGLE CRITIQUE : Jamais de WHERE company_id dans les models tenant.
COMMIT ATTENDU : feat: initialize Laravel 11 with multi-tenant PostgreSQL setup
```

PROMPT CC-02 — Module Auth complet (Semaine 2)

```
Tu développes le module d'authentification de Leopardo RH.

ENDPOINTS À IMPLÉMENTER (voir 02_API_CONTRATS.md) :
- POST /api/v1/auth/login
- POST /api/v1/auth/logout
- GET  /api/v1/auth/me
- POST /api/v1/auth/forgot-password
- POST /api/v1/auth/reset-password
- POST /api/v1/auth/device/fcm
- DELETE /api/v1/auth/device/fcm

POINTS CRITIQUES :
1. La table user_lookups (schéma public) mappe email → company_id → schema_name
```

```
   → À chaque login, chercher dans user_lookups, switcher le schéma, PUIS valider
le password
2. Le token Sanctum doit inclure le company_uuid dans les métadonnées
3. FCM token : stocker dans employee_devices (schema tenant), un par appareil
4. GET /auth/me doit retourner le profil complet + infos company (voir
API_CONTRATS.md)

TESTS OBLIGATOIRES (Pest) :
- test: login with valid credentials returns token
- test: login with wrong password returns 401
- test: employee A token cannot access company B data (TENANT ISOLATION)
- test: logout invalidates token

COMMIT ATTENDU : feat: add complete auth module with FCM device registration
```

PROMPT CC-03 — Module Pointage (Semaine 3)

```
Tu développes le module de pointage (attendance) de Leopardo RH.

ENDPOINTS À IMPLÉMENTER :
- POST /api/v1/attendance/check-in
- POST /api/v1/attendance/check-out
- GET  /api/v1/attendance/today
- GET  /api/v1/attendance/{employee_id}
- POST /api/v1/attendance/qr-scan
- POST /api/v1/attendance/biometric-webhook

RÈGLES MÉTIER (voir 09_REGLES_METIER_COMPLETES.md) :
1. TOUJOURS utiliser now() côté serveur pour check_in et check_out. JAMAIS le
timestamp client.
2. check-out = UPDATE sur la ligne existante, JAMAIS un INSERT.
3. Validation GPS : si le site a des coordonnées, vérifier distance <
site.gps_radius_meters
4. Statut automatique : ontime / late / early_leave / absent calculé selon
schedule de l'employé
5. Si employé déjà pointé arrivée → check-in retourne 422 (conflict)
6. GET /attendance/today : retourne null si pas encore pointé, le log sinon

TESTS OBLIGATOIRES :
- test: check-in creates attendance log with server timestamp
- test: second check-in returns 422 conflict
- test: check-out updates existing record, not creates new
- test: gps outside zone returns 422

COMMIT ATTENDU : feat: add attendance module with GPS validation and server-side
timestamps
```

PROMPT CC-04 — Module Absences (Semaine 5)

```
Tu développes le module de gestion des absences.

ENDPOINTS À IMPLÉMENTER :
- GET  /api/v1/absences
- POST /api/v1/absences
```

```
- GET  /api/v1/absences/{id}
- PUT  /api/v1/absences/{id}/approve
- PUT  /api/v1/absences/{id}/reject
- PUT  /api/v1/absences/{id}/cancel
- CRUD /api/v1/absence-types

RÈGLES MÉTIER :
1. Acquisition mensuelle de congés : voir REGLES_METIER_COMPLETES.md (ex: 2.5
j/mois)
2. Approbation → déduire du leave_balance de l'employé
3. Annulation → reccréditer le solde
4. Report annuel à configurer par entreprise (max_carry_over_days)
5. Notification push à l'employé quand statut change (Firebase)

COMMIT ATTENDU : feat: add absence module with balance management and push
notifications
```

PROMPT CC-05 — Module Paie (Semaine 7-8)

```
Tu développes le module de paie. C'est le module le plus critique en calculs.

ENDPOINTS À IMPLÉMENTER :
- POST /api/v1/payroll/calculate
- POST /api/v1/payroll/validate
- GET  /api/v1/payroll
- GET  /api/v1/payroll/{id}
- GET  /api/v1/payroll/{id}/pdf
- GET  /api/v1/payroll/status/{job_id}
- POST /api/v1/payroll/export-bank

RÈGLES MÉTIER (voir 09_REGLES_METIER_COMPLETES.md) :
1. Formule : Net = Brut - Cotisations salariales - IR - Retenues diverses
2. Les taux varient par pays (settings.hr_model)
3. Avances en cours → déduire amount_remaining, mettre à jour le statut de
l'avance
4. Absences non justifiées → déduire du salaire au prorata
5. Heures supplémentaires → ajouter selon taux configuré
6. La génération PDF est un job async (queue) → retourner job_id immédiatement

TESTS OBLIGATOIRES :
- test: payroll net = brut - cotisations - IR - retenues (valeurs exactes)
- test: advance deduction updates advance amount_remaining
- test: advance fully repaid sets status to repaid

COMMIT ATTENDU : feat: add payroll calculation with async PDF generation
```

□ JULES — Flutter Mobile

PROMPT JU-01 — Initialisation Flutter (Sprint 0, Semaine 1)

```
Tu es le développeur mobile principal de Leopardo RH. Tu initialises l'application
Flutter.

STACK OBLIGATOIRE (non négociable) :
- Flutter 3.x (stable channel)
- Riverpod 2.x (UNIQUEMENT — pas de BLoC, pas de Provider legacy)
- GoRouter 13.x
- Dio 5.x avec intercepteurs
- flutter_secure_storage (token Bearer)
- firebase_messaging, local_auth, geolocator, mobile_scanner
- flutter_localizations + intl (ARB files) — 4 langues : fr, ar, en, es

ÉTAPES :
1. flutter create --org com.leopardo leopardo_rh_mobile
2. Configurer pubspec.yaml (voir SPRINT_0_CHECKLIST.md section 0-D.2)
3. Configurer GoRouter avec les routes définies dans FEUILLE_DE_ROUTE.md
4. Créer AuthNotifier (Riverpod) avec état : loading / authenticated /
unauthenticated
5. Implémenter l'écran de login avec Dio pointant sur MOCK JSON (pas l'API réelle)
6. Configurer flutter_secure_storage pour stocker le token

RÈGLE : Jamais de String hardcodée dans l'UI. Toujours context.l10n.xxxKey
RÈGLE : Jamais de Dio direct dans un Widget. Toujours via Repository.
RÉFÉRENCE : Lire 06_PROMPT_MASTER_JULES_FLUTTER.md en entier avant de commencer.

COMMIT ATTENDU : feat: initialize Flutter app with GoRouter, Riverpod, auth flow
```

PROMPT JU-02 — Écran Pointage (Semaine 3)

```
Tu développes l'écran de pointage — le cœur de l'application.

ÉCRANS À CRÉER :
- HomeScreen (accueil employé) : grand bouton Pointer arrivée/départ, statut du
jour
- AttendanceConfirmScreen : animation succès après pointage
- AttendanceHistoryScreen : calendrier mensuel avec couleurs par statut

FLUX EXACT :
1. Au démarrage, appeler GET /attendance/today (ou mock_attendance_today.json)
   → si null → afficher bouton 'Pointer mon arrivée'
   → si check_in présent et check_out null → bouton 'Pointer mon départ'
   → si check_out présent → afficher résumé (heures travaillées, statut)
2. Bouton check-in → demander permission GPS → POST /attendance/check-in
3. En cas d'erreur GPS_ZONE → afficher message 'Vous n'êtes pas sur site'
4. Afficher TOUJOURS l'heure retournée par le serveur, pas DateTime.now()

MODÈLE DART À UTILISER : AttendanceLog (voir 20_MODELES_DART.md)

COMMIT ATTENDU : feat: add attendance check-in/out screens with GPS validation
```

PROMPT JU-03 — Connexion API Réelle (après backend prêt)

```
Le backend est maintenant déployé. Tu passes du mock JSON à l'API réelle.

ÉTAPES :
1. Modifier AttendanceRepository pour pointer sur https://api.leopardo-rh.com/v1
2. Implémenter le DioInterceptor : injecter Bearer token à chaque requête
3. Implémenter la gestion des erreurs API :
   - 401 → déconnecter l'utilisateur et rediriger vers login
   - 422 → afficher le message d'erreur du champ concerné
   - 503 → afficher 'Serveur indisponible, réessayez dans quelques instants'
4. Tester le flux complet sur un appareil Android physique
5. Vérifier que le token FCM est bien envoyé à POST /auth/device/fcm au login

COMMIT ATTENDU : feat: switch from mock to live API with DioInterceptor auth
```

□ CURSOR — Frontend Vue.js 3 (Gestionnaire Web)

PROMPT CU-01 — Dashboard Gestionnaire (Semaine 8)

```
Tu développes l'interface web gestionnaire de Leopardo RH avec Vue.js 3 +
Inertia.js.

STACK (déjà installée par Claude Code dans le projet Laravel) :
- Vue.js 3 + Inertia.js + Vite
- Tailwind CSS + PrimeVue (composants)

PAGES À CRÉER (Phase 1 — semaine 8-10) :
- /dashboard : KPIs du jour (présents, absents, en retard, tâches urgentes)
- /employees : liste + recherche + filtres département
- /employees/{id} : fiche employé complète
- /employees/create : formulaire création employé
- /attendance : liste pointages du jour + calendrier hebdomadaire

RÈGLES :
1. Les données viennent des Controllers Laravel via Inertia::render() — pas d'API
fetch direct
2. Utiliser les composants PrimeVue (DataTable, Calendar, Button, InputText)
3. Responsive obligatoire (le gestionnaire peut être sur tablette)
4. Respecter le RBAC : masquer les actions selon le manager_role (voir
10_RBAC_COMPLET.md)

RÉFÉRENCE : Wireframes dans leopardo_rh_wireframes_COMPLET.html
COMMIT ATTENDU : feat: add manager dashboard with attendance and employee list
```

□ V0.DEV (Vercel) — Landing Page & UI Prototyping

PROMPT VO-01 — Landing Page Leopardo RH

```
Crée la landing page commerciale de Leopardo RH. Elle sera intégrée dans Laravel
Blade.

SECTIONS REQUISES (dans l'ordre) :
1. Hero : 'Gérez vos RH simplement — Pointage, Absences, Paie en un clic'
   - Bouton CTA principal : 'Essai gratuit 14 jours'
   - Bouton secondaire : 'Voir une démo'
   - Mockup de l'app mobile visible à droite

2. Problématique : 3 icônes avec pain points (papier, erreurs de paie, retards)

3. Fonctionnalités : 4 piliers avec icônes et descriptions brèves
   - Pointage GPS + Biométrique ZKTeco
   - Gestion absences et congés
   - Paie automatisée avec PDF
   - Tâches et performance
```

```
4. Plans tarifaires : Starter (Gratuit/5 emp), Business (XX€/mois), Enterprise
(Sur devis)

5. FAQ : 6 questions sur sécurité, multi-pays, ZKTeco, mobile

6. Footer avec liens Contact, Blog, CGU, Politique de confidentialité

STYLE : Professionnel, couleurs bleu foncé (#1A5276) + orange (#E67E22). Tailwind
CSS.
FORMAT ATTENDU : HTML + Tailwind CSS (fichier unique) à intégrer dans Blade
```

□ MANUS — Agent Autonome (Tâches de génération de masse)

PROMPT MA-01 — Génération des fichiers Mock JSON Flutter

```
Tu es un agent autonome. Tu dois créer 8 fichiers JSON de mock data pour l'app
Flutter.
Ces fichiers doivent être EXACTEMENT conformes aux réponses API documentées dans
02_API_CONTRATS.md.

FICHIERS À CRÉER dans mobile/assets/mock/ :

1. mock_auth_login.json → Réponse POST /auth/login
   → Inclure : token, user (id, first_name, last_name, email, role, photo_url),
company

2. mock_auth_me.json → Réponse GET /auth/me
   → Même structure que user dans le login + leave_balance

3. mock_attendance_today.json → Réponse GET /attendance/today
   → Créer 2 versions : (a) data: null (pas encore pointé), (b) data: {check_in
présent}

4. mock_attendance_history.json → Liste de 30 entrées de pointage
   → Varier les statuts : ontime (60%), late (20%), absent (15%), early_leave (5%)

5. mock_absences.json → GET /absences + solde
   → 5 absences avec statuts variés (pending, approved, rejected)

6. mock_tasks.json → GET /tasks
   → 8 tâches avec priorités et statuts variés

7. mock_payroll.json → 6 derniers bulletins
   → Brut 45000 DA, taux cotisations Algérie, net calculé correctement

8. mock_notifications.json → 5 notifications, 2 non lues
   → Types : absence_approved, task_assigned, payslip_available

IMPORTANT : Les dates doivent être en ISO 8601. Les montants en décimaux avec
point.
```

PROMPT MA-02 — Génération des modèles Dart complets

```
Tu crées le fichier complet 20_MODELES_DART.md avec toutes les classes Dart.

CLASSES À GÉNÉRER (dans lib/shared/models/) :
- employee.dart : Employee (fromJson, toJson, getter fullName)
- attendance_log.dart : AttendanceLog (checkIn/checkOut TOUJOURS DateTime, jamais
String)
- absence.dart : Absence (avec AbsenceStatus enum)
- salary_advance.dart : SalaryAdvance (avec AdvanceStatus enum)
- task.dart : Task (avec TaskStatus et TaskPriority enums)
- payroll_slip.dart : PayrollSlip (tous les champs de déduction)
```

```
- app_notification.dart : AppNotification (avec NotificationType enum)
- project.dart : Project
- evaluation.dart : Evaluation

RÈGLES DART :
1. Tous les champs nullable avec ? explicite
2. Dates : DateTime.parse(json['field']) — jamais String
3. Montants : .toDouble() — jamais .toString()
4. Enums avec fromString() factory
5. copyWith() sur chaque classe

FORMAT : Fichier Dart complet par classe, prêt à copier-coller.
```

PROMPT MA-03 — Création des fichiers CI/CD GitHub Actions

```
Tu crées les 2 fichiers YAML de GitHub Actions pour le monorepo Leopardo RH.

FICHIER 1 : .github/workflows/tests.yml
- Déclenché à chaque Pull Request vers develop ou main
- Jobs backend : PHP 8.3, Composer, PostgreSQL 16, Redis 7, php artisan test --
parallel
- Jobs mobile : Flutter stable, flutter test
- Les deux jobs doivent passer pour autoriser le merge

FICHIER 2 : .github/workflows/deploy.yml
- Déclenché uniquement sur push sur main
- Connexion SSH o2switch via secrets (O2SWITCH_HOST, O2SWITCH_USER,
O2SWITCH_SSH_KEY)
- git pull → composer install --no-dev → php artisan migrate --force
- php artisan config:cache, route:cache, view:cache
- sudo supervisorctl restart all (pour Horizon queue workers)
- Notification Slack en cas d'échec (optionnel)

COMMIT ATTENDU : chore: add CI/CD GitHub Actions for tests and o2switch deployment
```

4. PROTOCOLE DE COMMUNICATION INTER-AGENTS

Comment les agents se coordonnent

SITUATION	QUI AGIT	COMMENT
Claude Code modifie un endpoint	Claude Code	Met à jour 02_API_CONTRATS.md + commit docs: update API contract for /attendance
Jules a besoin d'un endpoint non documenté	Toi (humain)	Ajouter le contrat dans 02_API_CONTRATS.md, puis redéclencher Jules
Un bug est découvert en prod	Claude Code	Branch hotfix/nom-du-bug depuis main, fix, merge vers main + develop
v0.dev génère un composant UI	Cursor	Intègre le HTML/CSS généré dans le composant Vue.js approprié
Manus génère les mocks JSON	Jules	Valide que les mocks correspondent aux wireframes avant de l'utiliser

Fichiers que chaque agent NE DOIT PAS modifier

AGENT	FICHIERS INTERDITS (ownership d'un autre agent)
Jules	api/* (tout le dossier backend)
Claude Code	mobile/* (tout le dossier Flutter)
Cursor	mobile/* + architecture backend (peut toucher les views Blade/Vue uniquement)
v0.dev	Tout fichier PHP, Dart ou logique métier
Manus	Tout fichier de code source. Manus génère uniquement des fichiers de données (JSON, YAML, MD)

5. QUICK REFERENCE — DÉMARRAGE IMMÉDIAT

Ordre des 5 premières actions à faire MAINTENANT

#	QUI	ACTION
1	Claude.ai (toi ici)	Compléter 02_API_CONTRATS.md avec auth/me + attendance/today + fcm endpoints
2	Manus	Générer les 8 fichiers mock JSON (Prompt MA-01)
3	Manus	Générer 20_MODELES_DART.md complet (Prompt MA-02)
4	Claude Code	Initialiser le projet Laravel 11 (Prompt CC-01)
5	Jules	Initialiser le projet Flutter (Prompt JU-01)

✅RAPPEL CRITIQUE : Donnez à chaque agent son prompt d'exécution (Section 3) ET les documents de référence listés dans ce prompt. L'agent ne peut pas bien travailler sans le contexte complet.

 Documents à joindre systématiquement à Claude Code : 01_PROMPT_MASTER.md, 02_API_CONTRATS.md, 07_SCHEMA_SQL.sql, 09_REGLES_METIER.md

 Documents à joindre systématiquement à Jules : 06_PROMPT_JULES.md, 02_API_CONTRATS.md, 20_MODELES_DART.md, mock JSON correspondants